

MYSQL VE PHP

7

PDO VE MYSQLI

PHP'de MySQL veri tabanına erişirken, iki seçeneğimiz vardır: **MySQLi** ve **PDO**. Peki, birini seçmeden önce ne bilmeliyiz?

Özet:

	MySQLi	PDO
Database desteği	Sadece MySQL	12 farklı database
API	OOP + procedural	OOP
Bağlanma	Kolay	Kolay
Parametre isimlendirme	Hayır	Evet
Object mapping	Evet	Evet
Hazırlanan ifadeler (clientside)	Hayır	Evet
Performans	Hızlı	Hızlı
Storedprosodür	Evet	Evet

PDO

SUNUCUYA BAĞLANMAK

Bu bölümde PHP Veri Nesnelerini veya PDO kullanarak MySQL ile nasıl etkileşimde bulunacağınızı öğreneceksiniz.

PDO bir veri erişim soyutlama katmanıdır. MySQL de dahil olmak üzere herhangi bir veri tabanı ile etkileşim kurmak için hafif ve tutarlı bir ara yüz sağlayan bir PHP uzantısıdır.

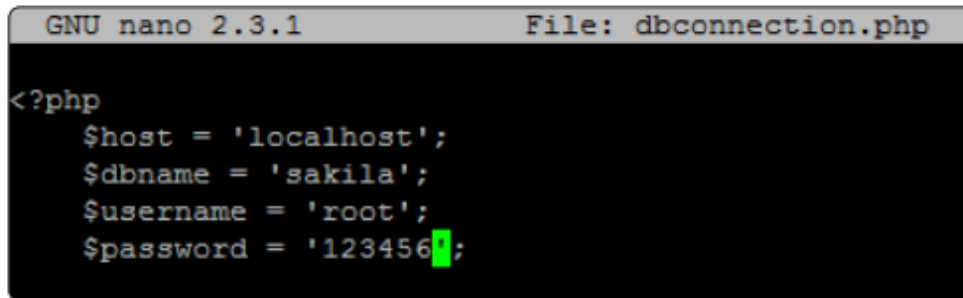
Bir MySQL veritabanına bağlanmadan önce aşağıdaki bilgileri belirtmeniz gerekir:

- **Host:** <mysqlserver bulunduğu adres>
- **Dbname:** <database ismi>
- **Username:** <MySQL kullanıcısı>
- **Password:** <Kullanıcının şifresi>

Linux Sunucumuzun </var/www/html/> yoluna;

```
# nano dbconnection.php
```

PHP dosyasının içine db bağlanmak için değişkenlerimizi tanımlıyoruz.



```
GNU nano 2.3.1 File: dbconnection.php

<?php
    $host = 'localhost';
    $dbname = 'sakila';
    $username = 'root';
    $password = '123456';
```

Veritabanı Bağlantı Bilgileri

```
# nano index.php
```

```

GNU nano 2.3.1      File: index.php      Modified
<?php
require_once 'dbconnection.php';

try {
    $conn = new PDO("mysql:host=$host;dbname=$dbname", $username, $password);
    echo "Bağlantı başarılıdır...";
} catch (PDOException $pe) {
    die("Database bağlantı yapılamıyor.. : " . $pe->getMessage());
}

```

PDO ile Veri tabanı Bağlantısı

Bağlantı başarılıdır...

Bağlantı Başarı ile Yapıldı

MySQLi ile Bağlanma:

```

// mysqli, procedural
$mysqli = mysqli_connect('localhost','username','password','database');

// mysqli, objectoriented
$mysqli = new mysqli('localhost','username','password','database');

```

```

GNU nano 2.3.1      File: index.php
<?php
require_once('dbconnection.php');
$mysqli = new mysqli($host,$username,$password,$dbname);
var_dump($mysqli);

```

MySQLi ile Bağlantı

Hem PDO hem de MySQL'i, nesne yönelimli bir API sunar, ancak MySQL'i ayrıca usule dayalı bir API sunar; bu da yeni gelenlerin anlamasını kolaylaştırır. Yerli PHP MySQL sürücüsünü biliyorsanız, prosedürel MySQL'i arayüzüne geçiş işlemi çok daha kolay bulacaksınız. Öte yandan, PDO'ya hakim olduktan sonra onu istediğiniz herhangi bir veritabanı ile kullanabilirsiniz!



NOT Karakter setleri hakkında bir hata alırsanız, charset parametresini DSN'ye eklediğinizden emin olun. Karakter setini DSN'ye eklemek güvenlik nedenlerinden ötürü önemlidir.



Buradan itibaren sadece PDO ilgili anlatımlarımız olacaktır.

```
<?php
$db = new PDO('mysql:host=localhost;dbname=sakila;charset=utf8mb4',
'username', 'password');
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$db->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);
```

Hata Yönetimi:

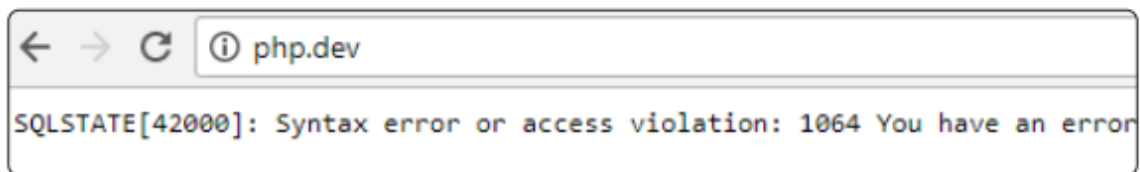
```
$db = new PDO('mysql:host=localhost;dbname=sakila;charset=utf8mb4',
'root', '');
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$db->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);
try{
$db->query('selam');

}catch(PDOException $e){

echo $e->getMessage();

}
```

Çıktı:



Bağlantı Hatası

Tablo (Grid) Döndüren Sorgularla Çalışmak:

```
foreach($db->query('SELECT * FROM actor') as $row) {
echo $row['actor_id'].' '.$row['first_name'].'<hr/>';
}
```

Aşağıdaki kodda aynı sonucu verecektir.

```
foreach($db->query('SELECT * FROM actor') as $row) {
    echo $row[0].' '.$row[1].<hr/>';
}
```

query () yöntemi bir **PDOStatement** nesnesi döndürür. Sonuçları şu şekilde de getirebilirsiniz:

```
while($row = $stmt->fetch(PDO::FETCH_ASSOC)) {
    echo $row['actor_id'].' '.$row['first_name'];
}
```

FetchModes(Getirme Metotları): Yukarıdaki **fetch ()** ve **fetchAll ()** kodlarında **PDO :: FETCH_ASSOC** kullanımına dikkat edin. Bu, PDO'ya alan adlarını anahtar olarak ilişkisel bir dizi olarak döndürmesini söyler. **PDO :: FETCH_NUM** gibi diğer getirme modları, satırı sayısal bir dizi olarak döndürür. Varsayılan, verileri sayısal ve ilişkili tuşlarla çoğaltan **PDO :: FETCH_BOTH** ile getirmektir. İki veya daha fazla boyutta dizileriniz olmaması için birini veya birini belirtmenizi öneririz! PDO, **PDO :: FETCH_OBJ** ile nesneleri de getirebilir ve mevcut sınıfları **PDO :: FETCH_CLASS** ile alabilir. **PDO :: FETCH_BOUND** ve **bindColumn** yöntemini kullanarak belirli değişkenlere de bağlanabilir.

Döndürülen Satır sayısını Alma:

```
$stmt = $db->query('SELECT * FROM actor');
$row_count = $stmt->rowCount();
echo $row_count;
```

Çıktı: 200

Ekleme Sonrası Son ID Alma:

```
$result = $db->exec("INSERT INTO actor(first_name,last_name)
VALUES('Ahmet', 'Elma');");
$insertId = $db->lastInsertId();
echo $insertId;
```

Çıktı:201

UPDATE, DELETE, INSERT Yapılarında Etkilenen Satır Sayısını Döndürmek:

```
$etkilenen_satirlar = $db->exec("UPDATE actor SET last_update=NOW()
WHERE actor_id BETWEEN 1 AND 5");
echo $etkilenen_satirlar;
```

Çıktı:5**Adlandırılmış Parametreler (NamedParameters):**

Bu, PDO'nun sahip olduğu bir başka önemli özelliktir; bağlayıcı parametreler sayısal bağlamadan çok daha kolaydır.

Şu ana kadar yalnızca herhangi bir değişkene girmeyen basit ifadeler gösterdik. Bunlar basit ifadelerdir. Değişken parametrelerini kullanan ifadeler için, sorgularınızı güvenli yürütmek, bağlı parametre yöntemlerini kullanmalısınız.

```
$id=5;
$name='Ahmet';
$stmt = $db->prepare("SELECT * FROM actor WHERE actor_id=? OR first_
name=?");
$stmt->execute(array($id, $name));
$rows = $stmt->fetchAll(PDO::FETCH_ASSOC);
var_dump($rows);
```

Çıktı

```
array(3) {
  [0]=>
  array(4) {
    ["actor_id"]=>
    int(5)
    ["first_name"]=>
    string(6) "JOHNNY"
    ["last_name"]=>
    string(12) "LOLLOBRIGIDA"
    ["last_update"]=>
    string(19) "2017-12-14 12:23:34"
  }
  [1]=>
  array(4) {
```

```

        ["actor_id"]=>
int(201)
        ["first_name"]=>
string(5) "Ahmet"
        ["last_name"]=>
string(4) "Elma"
        ["last_update"]=>
string(19) "2017-12-14 11:52:01"
    }
    [2]=>
array(4) {
        ["actor_id"]=>
int(202)
        ["first_name"]=>
string(5) "Ahmet"
        ["last_name"]=>
string(4) "Elma"
        ["last_update"]=>
string(19) "2017-12-14 11:52:39"
    }
}

```

INSERT, UPDATE ve DELETE Parametre Atanması:

INSERT, UPDATE ve DELETE için hazırlanmış bildirimler SELECT'den farklı değildir. Ancak yine de bazı örnekler yapalım:

INSERT:

```

<?php
$name='Doğuhan';
$lastname='ELMA';
$stmt = $db->prepare("INSERT INTO actor(first_name,last_name)
VALUES(:name,:lastname)");
$stmt->execute(array(':name' => $name, ':lastname' => $lastname));
$affected_rows = $stmt->rowCount();

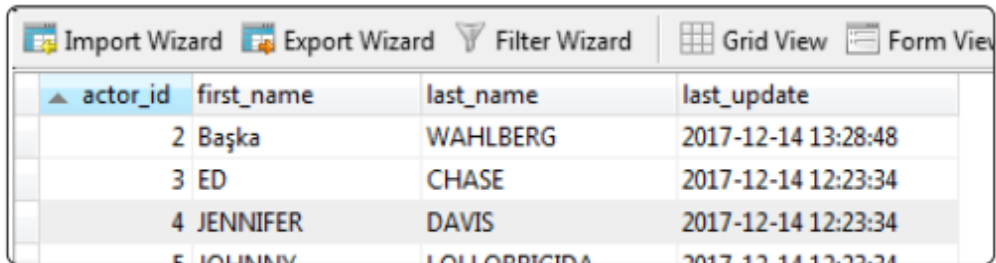
```

DELETE:

```
<?php
$id=1;
$stmt = $db->prepare("DELETE FROM actor WHERE actor_id=:id");
$stmt->bindValue(':id', $id, PDO::PARAM_STR);
$stmt->execute();
$affected_rows = $stmt->rowCount();
```

UPDATE:

```
<?php
$id=2;
$name='Başka';
$stmt = $db->prepare("UPDATE actor SET first_name=? WHERE actor_id=?");
$stmt->execute(array($name, $id));
$affected_rows = $stmt->rowCount();
```

Çıktı:


The screenshot shows a database application window with a toolbar at the top containing 'Import Wizard', 'Export Wizard', 'Filter Wizard', 'Grid View', and 'Form View'. Below the toolbar is a table with the following data:

actor_id	first_name	last_name	last_update
2	Başka	WAHLBERG	2017-12-14 13:28:48
3	ED	CHASE	2017-12-14 12:23:34
4	JENNIFER	DAVIS	2017-12-14 12:23:34
5	JOHNNY	LOLOBRIGIDA	2017-12-14 12:23:34

Transaction:

```
try {
    $db->beginTransaction();

    $db->exec("sorgu");

    $stmt = $db->prepare("diğer sorgu");
    $stmt->execute(array($value));

    $stmt = $db->prepare("başka bir sorgu");
    $stmt->execute(array($value2, $value3));
```



```

        $db->commit();
    } catch(PDOException $ex) {
        $db->rollBack();
        echo $ex->getMessage();
    }

```

CRONTAB

Soru: Sunucumda komut satırından yürütülebilen ve ayrıca Apache web sunucusunu kullanarak tarayıcıdan erişilebilen bir php komut dosyası var. Bu php komut dosyasını her gün yürütmek istiyorum. Bunu Linux ortamında phpcron işi olarak nasıl programlayabilirim?

Linux makinelerinde arka planda görevleri çalıştırmanın en standart yollarından biri cron işleriyle ilgilidir. Bunlar, VPS'deki görevleri planlamak ve farklı bakımla ilgili işleri otomatikleştirmek için kullanışlıdır. "Cron" kendisi arka planda çalışan bir arka plan programıdır (*veya programdır*). Çalıştırılan farklı işler için zamanlama, "crontab" adlı bir yapılandırma dosyasında bulunur.

Kurulum: Neredeyse tüm dağıtımların varsayılan olarak bir çeşit cron yüklüdür. Bununla birlikte, yüklü olmayan bir sistem kullanıyorsanız, bunu aşağıdaki komutlarla yükleyebilirsiniz:

```

#yum update
#yum installvixie-croncrontabs

```

Crontab'ın sistem yeniden başladığında başlayabilmesi için ayarlıyor ve o anki durumunu öğreniyoruz.

```
[root@elmssoft ~]# systemctl enable crond
[root@elmssoft ~]# service crond status
Redirecting to /bin/systemctl status crond.service
● crond.service - Command Scheduler
   Loaded: loaded (/usr/lib/systemd/system/crond.service; enabled; vendor preset: enabled)
   Active: active (running) since Mon 2017-12-11 00:17:50 +03; 3 days ago
 Main PID: 1272 (crond)
    CGroup: /system.slice/crond.service
            └─1272 /usr/sbin/crond -n

Dec 11 00:17:50 elmssoft systemd[1]: Started Command Scheduler.
Dec 11 00:17:50 elmssoft systemd[1]: Starting Command Scheduler...
Dec 11 00:17:50 elmssoft crond[1272]: (CRON) INFO (RANDOM_DELAY will be scaled...)
Dec 11 00:17:50 elmssoft crond[1272]: (CRON) INFO (running with inotify support)
Hint: Some lines were ellipsized, use -l to show in full.
[root@elmssoft ~]#
```

Crond Servisi Çalışma Durumu

```
#nano /etc/crontab
```

```
GNU nano 2.3.1      File: /etc/crontab

SHELL=/bin/bash
PATH=/sbin:/bin:/usr/sbin:/usr/bin
MAILTO=root

# For details see man 4 crontabs

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | .---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,$
# | | | | |
# * * * * * user-name  command to be executed

[ Read 15 lines ]
^G Get Help  ^O WriteOut  ^R Read File ^Y Prev Page ^K Cut Text  ^C Cur Pos
^X Exit      ^J Justify   ^W Where Is  ^V Next Page ^U UnCut Text ^T To Spell
```

Crontab Listesi

Görüldüğü gibi herhangi bir crontab eklenmemiştir.

Crontab Komutları:

```
#crontab -e
```

Crontab dosyasını oluşturur yada düzenler. Crontab varsayılan editörü '**vim**'dir.

Editörü değiştirmek için;

```
#export EDITOR=nano
```

```
#crontab -l
```

Geçerli crontab işlerini listeler.

```
#crontab -r
```

Bu kullanım ise tüm zamanlanmış görevleri siler.

Zamanlanmış Görev Oluşturma: Her dakika **"`/var/www/html/gorev.php`"** yolundaki php dosyasını çalıştıralım.

"gorev.php" İçeriği:

```
<?php
$db = new PDO('mysql:host=localhost;dbname=sakila;charset=utf8mb4',
    'root', '***');
$db->setAttribute(PDO::ATTR_ERRMODE, PDO::ERRMODE_EXCEPTION);
$db->setAttribute(PDO::ATTR_EMULATE_PREPARES, false);
try{
    $db->exec("insert into actor(first_name,last_name,last_update)
VALUES ('isim','soyisim',NOW())");
}catch(PDOException $e){

echo $e->getMessage();

}

#crontab -e

* * * * * php /var/www/html/gorev.php
```

Online hesaplayıcılarla daha doğru zamanlamalar oluşturabilirsiniz. <https://crontab.guru>

“At 01:05.”

next at 2017-12-16 01:05:00 random

5 1 * * *

minute	hour	day (month)	month	day (week)
*	any value			
,	value list separator			
-	range of values			
/	step values			
0-23	allowed values			

Crontab Zamanlama Online Servisi

Benim kullandığım açık bir servistir. Okla gösterilen kısım bir sonraki cron'un çalışma zamanını göstermektedir.

Sonuç:

▼ actor_id	first_name	last_name	last_update
207	isim	soyisim	2017-12-15 00:07:01
206	isim	soyisim	2017-12-15 00:06:01
205	isim	soyisim	2017-12-15 00:05:01
200	THORA	TEMPLE	2006-02-15 04:34:33
199	JULIA	FAWCETT	2006-02-15 04:34:33

Crontab Çıktısı

CrontabLog: `"/var/log/cron"` yolu crontablog kayıtlarının barındırıldığı yerdir.

PHP DOSYA - DİZİN İŞLEMLERİ

PHP ile sunucu üzerinde dosya/dizin oluşturmak için güçlü fonksiyonları mevcuttur. Örneklerimizde bir dosyayı sunucuya yüklemek (*upload*) ve sunucudan almayı ele alacağız.

Başlarken: PHP, dosya yüklemelerini işlemek için çok yönlü ve kullanıcı dostu bir arayüz sunsa da varsayılan yükleme, 2 Mega Byte'ı aşan dosyalarla çalışmak için tasarlanmamıştır. Bu makale, PHP motorunuzu böyle geniş dosya aktarımlarını gerçekleştirmek için yapılandırmanıza yardımcı olacaktır.

PHP.INI: Kurulumunuzun tüm yapılandırma ayarları php.ini dosyasında bulunur. Bazen bu ayar apache .htaccess dosyalarındaki direktiflerle veya komut dosyalarının kendisiyle bile geçersiz kılınabilir. Bu yüzden sadece ini dosyasına konsantre olalım. **Php.ini** dosyanızın bulunduğu yeri bulmak için **phpinfo ()** işlevini çağırabilir, ayrıca aşağıdaki ayarlarda değişiklik yapmak zorunda olduğumuz geçerli değerleri size gösterebilir.

- file_uploads
- upload_max_filesize
- max_input_time
- memory_limit
- max_execution_time
- post_max_size

upload_max_filesize ve post_max_size:

Dosyalar genellikle '**multipart/form-data**' olarak bilinen bir formatta web sunucusuna **POST** yapılır. **Post_max_size**, bir komut dosyasının bu şekilde kabul edebileceği veri miktarının üst sınırını ayarlar. İdeal olarak bu değer **upload_max_filesize** için ayarladığınız değerden daha büyük olmalıdır.

Upload_max_filesize ögesinin, yüklemekte olduğunuz tüm dosyaların boyutlarının toplamı olduğunu bilmek önemlidir. **Post_max_size**, **upload_max_filesize** artı formdaki diğer tüm alanların uzunlukları ve şifreleyicinin içerebileceği herhangi bir mime başlığının toplamıdır.

Memory_limit:

PHP motoru, gelen POST'u işlerken, gelen verilerin bir kısmını belleğinde tutması gerekir. Bu yönergenin yalnızca, yapılandırma sırasında **-enable-memory-limit** seçeneğini kullanmanız durumunda herhangi bir etkisi vardır. Çok yüksek bir değere ayarlamak çok tehlikeli olabilir, çünkü birkaç yükleme aynı anda ele alınırsa tüm kullanılabilir bellek tüketilir ve çok fazla bellek kullanan diğer ilgisiz komutlar tüm sunucuyu da etkileyebilir.

Max_execution_time ve max_input_time:

Bu ayarlar, komut dosyasının maksimum ömür süresini ve komut dosyasının girdiyi kabul etmede harcayacağı zamanı tanımlar. Birkaç mega bayt veri

```
appendfsynceverysec

# systemctlstartredis

# systemenableredis
```

REDIS KULLANIMI

Redis sunucusuna Bağlanma:

```
# redis-cli -a 123456
```

Değer Ayarlama ve Alma:

```
127.0.0.1:6379>set key01 value01
127.0.0.1:6379>get key01
```

Redis komutlarına aşağıdaki bağlantıdan bakabilirsiniz. <https://redis.io/commands>

REDIS VE PHP

PHP'deRedis'i kullanabilmek için;

```
# yum install php71w-pecl-redis
```

PHP uzantısını yüklememiz gerekir.

```
# nanoredis.php

<?php
$redis = newRedis();
$redis->connect("127.0.0.1",6379);
$redis->auth("123456");

// set ve getKey
$redis->set('key01', 'value01');
print 'key01.value : ' . $redis->get('key01') . "\n";
```

```

// append ve getKey
$redis->append('key01', 'value02');
print 'key01.value : ' . $redis->get('key01') . "\n";

$redis->set('key02', 1);
print 'key02.value : ' . $redis->get('key02') . "\n";

// increment
$redis->incr('key02', 100);
print 'key02.value : ' . $redis->get('key02') . "\n";

// decrement
$redis->decr('key02', 51);
print 'key02.value : ' . $redis->get('key02') . "\n";

// list
$redis->lPush('list01', 'value01');
$redis->rPush('list01', 'value02');
print 'list01.value : ';
print_r ($redis->lRange('list01', 0, -1));

// hash
$redis->hSet('hash01', 'key01', 'value01');
$redis->hSet('hash01', 'key02', 'value02');
print 'hash01.value : ';
print_r ($redis->hGetAll('hash01'));

// set
$redis->sAdd('set01', 'member01');
$redis->sAdd('set01', 'member02');
print 'set01.value : ';
print_r ($redis->sMembers('set01'));
?>

# phpredis.php

//Çıktılar Aşağıdadır.
key01.value : value01
key01.value : value01,value02
key02.value : 1

```

```

key02.value : 101
key02.value : 50
list01.value : Array
(
    [0] => value01
    [1] => value02
)
hash01.value : Array
(
    [key01] => value01
    [key02] => value02
)
set01.value : Array
(
    [0] => member02
    [1] => member01
)

```

NODE.JS

Node.js, sunucu tarafında programlama için bir Javascript platformudur. Kullanıcıların, arka uç işlevleri gerektiren ağa bağlı uygulamalar kolayca oluşturmalarına olanak tanır. Javascript'i istemci ve sunucu dili olarak kullanılır.

KURULUM

```
# yum --enablerepo=centos-scl-rh -y install rh-nodejs6
```

Ortam değişkenlerini yükleyelim:

```
# sclenable rh-nodejs6 bash
```

Versiyonunu görelim:

```
# node -v
```

```
V6.9.1
```

```
# which node
```

```
/opt/rh/rh-nodejs6/root/usr/bin/node
```

Sistem yeniden başladığında otomatik yüklenmesini sağlayalım:

```
# nano /etc/profile.d/rh-nodejs6.sh

#!/bin/bash
source /opt/rh/rh-nodejs6/enable
export X_SCLS=""`sclenable rh-nodejs6 `echo $X_SCLS`"
```

Merhaba Dünya:

```
# nano merhabadunya.js

var http = require('http');
http.createServer(function (req, res) {
  res.writeHead(200, {'Content-Type': 'text/plain'});
  res.end('Merhaba Dünya\n');
}).listen(4242, '127.0.0.1');
console.log('listening on http://127.0.0.1:4242/');
```

Sunucuyu çalıştıralım:

```
# node merhabadunya.js&
```

İstemcide çalıştıralım:

```
# curl http://127.0.0.1:4242/
```

SOHBET UYGULAMASI

Gerekli uzantı yüklemek:

```
# npm install -g expresssocket.io
```

Sunucu sohbet Script yazalım:

```
# nano sohbet.js

var app = require('express')();
var http = require('http').Server(app);
var io = require('socket.io')(http);
app.get('/', function(req, res){
```

```

res.sendFile(__dirname + '/index.html');
});

io.on('connection', function(socket){
socket.on('chatmessage', function(msg){
io.emit('chatmessage', msg);
});
});

http.listen(4242, function(){
console.log('çalışıyor... *:4242');
});

```

İstemci için;

```
# nano index.html
```

```

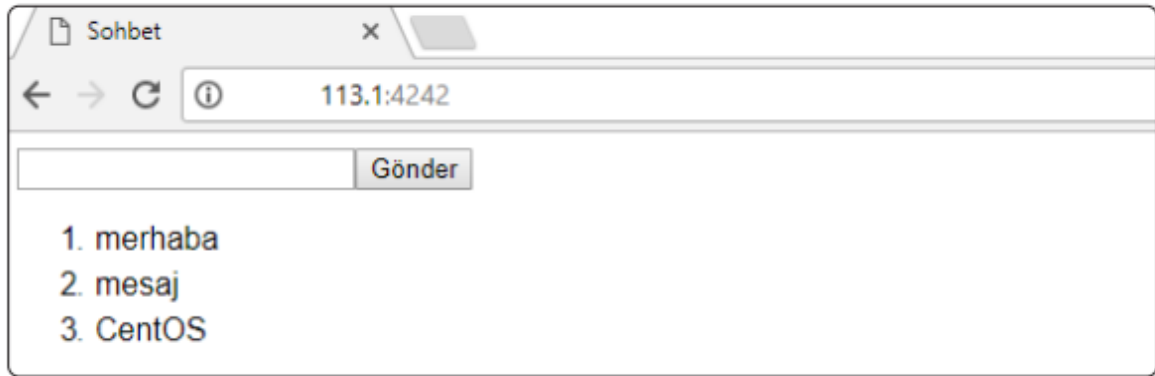
<!DOCTYPE html>
<html>
<head><title>Sohbet</title></head>
<body>
<form action="">
<input id="sendmsg" autocomplete="off" /><button>Gönder</button>
</form>
<ul id="messages" style="list-style-type: decimal; font-size: 16px">
<script src="/socket.io/socket.io.js"></script>
<script src="http://code.jquery.com/jquery.min.js"></script>
<script>
var socket = io();
$(`form`).submit(function(){
socket.emit('chatmessage', $('#sendmsg').val());
$('#sendmsg').val('');
return false;
});
socket.on('chatmessage', function(msg){
$(`#messages`).append($(`<li style="margin-bottom: 5px;">`).
text(msg));
});
</script>
</body>
</html>

```

Sunucuyu çalıştıralım:

```
# node sohbet.js
```

```
Çalışıyor... *:4242
```

İstemciyi çalıştıralım:

Sohbet Mesajları